

Applied-Qual-Fall-2024-solutions

October 7, 2024

1 Grading guide

Grade each problem out of 10 marks, making the entire test out of 60.

I propose that, for each problem:

- 9 or 10 represents an “A” level grade (“excellent work”);
- 7 or 8 represents a “B” level grade (“passing work”);
- 6 is a C (that being a failing grade in graduate school).

This would imply that the passing grade for the exam is roughly 40 out of 60. We may adjust this downwards slightly if we decide, after the fact, that the exam was holistically harder than the sum of its parts - i.e. the two graders should each grade independently, meet to discuss the results, and then decide on what the pass threshold should be.

I will give some guidance as to what grades of A and B represent on these problems; of course, there may be other ways of answering the problems that I don’t anticipate.

Note: several places I say things like “the code runs” and what I really mean is that “it ran on the student’s machine and produced viable output”. There’s always some chance that the code does something weird on our computers, despite it working fine on a student’s computer, due to some configuration issue. Before penalizing a student for code not running on the grader’s computer, it’s important to think through carefully why the problem has happened, and to give the student the benefit of the doubt.

Graders should feel free to unit test bits of student code, if it’s helpful to understand what the student has done, of course.

2 Instructions

- **This is a take-home exam.** Attempt all 6 problems; submit your answers by editing them into this jupyter notebook, running them, and emailing the completed notebook to Benjamin Young by the deadline (48 hours after the exam was made available).
- **This is an open-book exam.** You may use whatever reference materials you wish, including existing materials on the internet, your textbooks, previous problem sets, and your notes.
- **Please cite your sources** in your responses to each problem.
- **Please work on your own** and please don’t post the exam online or otherwise ask for help on these problems.

- Please explain your answers clearly in a markdown cell, in addition to any code you write. Before handing in your code, please run it yourself - ideally, the graders of this exam shouldn't need to rerun your code (we *may* do that, in an attempt to figure out what you've done or some such thing, but we shouldn't *need* to).

3 Problem 1 - Airy Equation

This ODE is known as the Airy equation:

$$\frac{d^2 y}{dx^2} - xy = 0.$$

One of the solutions is the [Airy function](#) $Ai(x)$, which has the property that

$$\lim_{x \rightarrow \infty} Ai(x) = 0.$$

If we also specify values $y(0), y'(0)$, then we have an initial value problem. Write code to solve this initial value problem numerically, using an appropriate finite-difference technique; your solution should take in values for $y(0), y'(0)$, a step size, and a maximum x value, $x_{\max}$; it will produce a list of values for a function y which approximately solves the *DE* on the interval $[0, x_{\max}]$. Plot your solution when $y(0) = 1, y'(0) = 0$.

Describe the error analysis of the method you have implemented. For instance, if you've used a Taylor series method, you should be able to give a big O bound on the error introduced in one step of the method. Other methods of solution may have different types of error analysis.

I imagine that none of the numerical solutions you find will have the above asymptotic property, even if you use initial values that are close to those of $Ai(x)$ - rather, I imagine that the solution will almost always blow up as x gets large. Explain why.

4 Solution

One thing we could use here is the forward Euler method: We pick a step size h , write $x_i = ih$ (so that $x_0 = 0$ is the initial condition) and approximate points y_i , using

$$\frac{d^2 y}{dx^2} \approx \frac{y_{i+2} - 2y_{i+1} + y_i}{h^2}$$

we get

$$\frac{y_{i+2} - 2y_{i+1} + y_i}{h^2} - iy_i = 0$$

or, rearranging,

$$y_{i+2} = (ih^3 - 1)y_i + 2y_{i+1}$$

.

This gives a second-order linear recurrence, allowing us to compute as many of the points y_i as we like, given the first two points. To approximate the initial conditions $y(0) = 1, y'(0) = 0$ we could use (say) $y_0 = y_1 = 1$. Let's for example do this using $h = 0.01$ (I choose it arbitrarily).

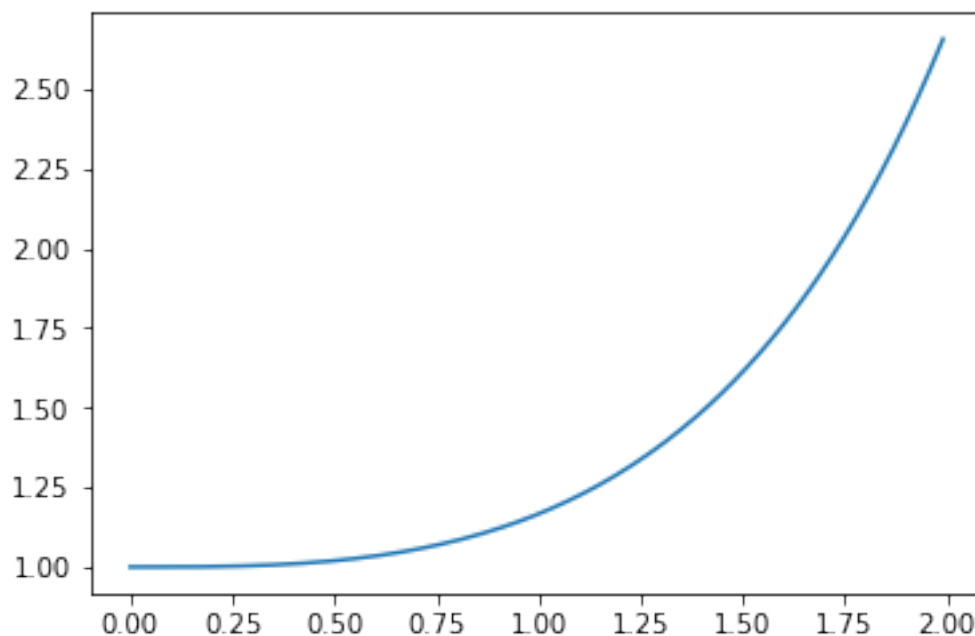
```
[4]: import numpy as np
import matplotlib.pyplot as plt

h = 0.01
x = np.arange(0, 2, h)
y = [1, 1]
n = len(x)-2

for i in range(n):
    y.append((i*h*h*h-1) * y[i] + 2*y[i+1])

plt.plot(x,y)
```

[4]: [<matplotlib.lines.Line2D at 0x71cfa6a38c88>]



To analyze the error, we write

$$y(x+h) = y(x) + hy'(x) + \frac{h^2}{2}y''(x) + \frac{h^3}{6}y'''(x) + O(h^4)$$

$$y(x-h) = y(x) - hy'(x) + \frac{h^2}{2}y''(x) - \frac{h^3}{6}y'''(x) + O(h^4)$$

Adding these two and rearranging, we learn that

$$\frac{y(x+h) - 2y(x) + y(x-h)}{h^2} = y''(x) + O(h^2)$$

So, one would expect roughly $O(h^2)$ error with each step of the method. Of course, this is an iterative method, so the errors blow up; indeed, the above function looks nothing like the Airy function, and the solution blows up as advertised. This is unsurprising: The link above, or any reference on the Airy function, will tell you that the Airy equation, being a second-order linear ODE, has a two-parameter family of solutions:

$$C_1 \text{Ai}(x) + C_2 \text{Bi}(x).$$

Moreover, A_i goes to zero as x gets large, whereas B_i blows up exponentially. In order for such a linear combination NOT to blow up, C_2 would have to be zero - i.e. the generic behavior of any true solution is exponential growth as x gets large. Since we're finding an approximate solution, it likely that our solution will exhibit this generic behavior.

5 Grading

To get a grade in the A range (9 or 10): the code should run, there should be some attempt at error analysis of the method, and some discussion of the “blow-up” behavior of the solution.

To get a grade in the B range (7 or 8): the code still runs but some aspect of the analysis is missing. Working code is essential.

6 Problem 2 - Wave Equation

This problem asks you to solve the [wave equation in one spatial dimension](#):

$$\frac{\partial^2 u}{\partial x^2} = \frac{\partial^2 u}{\partial t^2}$$

on the time domain $[0, 2\pi]$ and space domain $[0, 2\pi]$. Evidently, one solution to this equation is

$$U(t, x) = \sin(t) \sin(x).$$

Verify that knowing - $u(0, x) = U(0, x)$ and $u_t(0, x) = U_t(0, x)$ for $0 \leq x \leq 2\pi$, - $u(t, 0) = U(t, 0)$ and $u(t, 2\pi) = U(t, 2\pi)$ for $0 \leq t \leq 2\pi$

is sufficient information to reproduce the solution $u = U$ numerically.

Then, solve this IVP numerically; compare your solution to the exact solution. Any method you used in class is appropriate.

7 Solution

The desired solution $u = u_{\text{ref}}$ has $u(0, x) \equiv 0 \equiv u(t, 0)$; moreover, we compute $u_t(t, x) = \cos(t) \sin(x)$, so $u_t(0, x) = \sin(x)$; similarly, $u_x(t, 0) = \sin(t)$.

Much as in the above problem, we use a forward Euler-like scheme. Choose a step size h , and choose a grid of points (x_i, t_j) with spacing h , so that $x_i = ih, t_j = jh$. We then use the approximations

$$\frac{\partial^2 u}{\partial x^2}(x_i, t_j) \approx \frac{u((i+1)h, jh) - 2u(ih, jh) + u((i-1)h, jh)}{h^2}$$

and similarly

$$\frac{\partial^2 u}{\partial x^2}(x_i, t_j) \approx \frac{u(ih, (j+1)h) - 2u(ih, jh) + u(ih, (j-1)h)}{h^2}$$

Setting these two equal and rearranging, we find that (for instance) we can write

$$u((i+1)h, jh) \approx u(ih, (j+1)h) + u(ih, (j-1)h) - u((i-1)h, jh)$$

So, if we know values for $u(0, jx)$ and $u(h, jx)$ for “all” $0 \leq j \leq n$, we can find values $u(2h, jx)$ for $0 < j < n$, but we’d need to know $u(2h, 0)$ and $u(2h, nh)$ from some other explicit information - say, the boundary values of the desired solution.

To get the first two pieces of information from $u(0, x)$, we can again use the Taylor series / linear approximation,

$$u(h, jh) \approx u(0, jh) + hu'(0, jh)$$

So, that’s one set of initial/boundary values that would work for us to solve this problem numerically: values of u and u_t when $t = 0$, and values of u when $x = 0, 2\pi$.

```
[71]: n = 100
      pi = 3.14159
      h = 2.0*pi / n

      u_0 = np.asarray([0 for i in range(n+1)])
      ut_0 = np.asarray([np.sin(i*h) for i in range(n+1)])
      u_1 = u_0 + h*ut_0

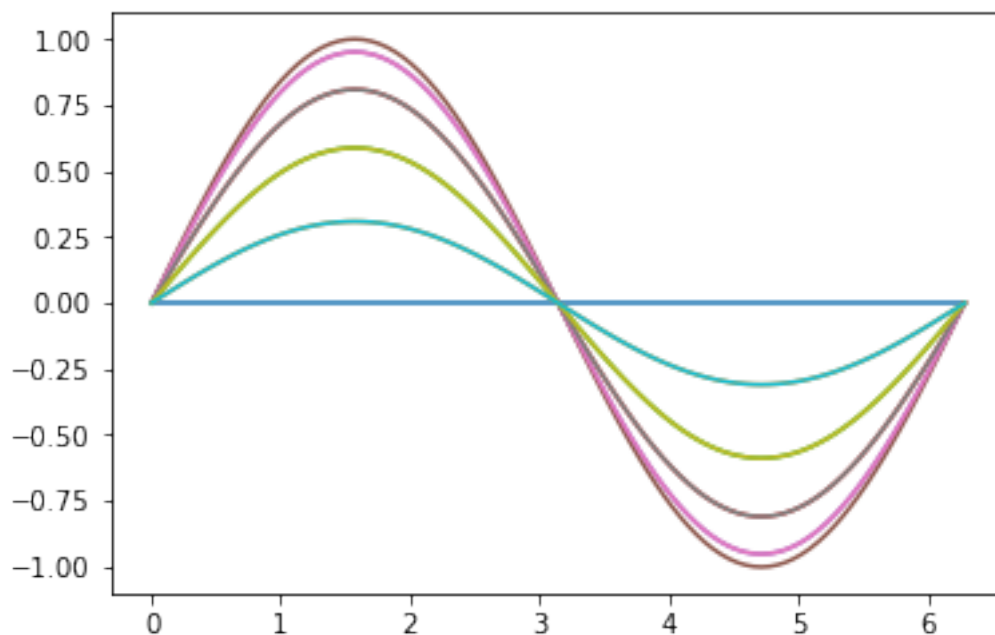
      u = [u_0, u_1]

      for i in range(1, n-1):
          work = [None] * (n+1) # work will become u[i+1]
          work[0] = 0
          work[n] = 0
          for j in range(1, n):
              work[j] = u[i][j+1] + u[i][j-1] - u[i-1][j]
          u.append(np.asarray(work))

      t = np.arange(0, 2*pi+h, h)
      x = np.arange(0, 2*pi+h, h)

      for j in range(0, 50, 5):
```

```
plt.plot(t, u[j])
```



Above is the plot of a few cross-sections of the computed solution.

8 Grading

To get a grade in the A range: the code runs and produces something close to the desired solution, and there is a discussion of why the described initial/boundary value data suffices to find a solution.

To get a grade in the B range: the code runs but something's off about the solution, or discussion is absent. Again, working code is crucial.

9 Problem 3 - Call Center Statistics

Suppose that a call center expects to receive on average λ calls per minute. Assuming that the timing of these calls is random and uncorrelated, the number of calls k received per minute, where k is a nonnegative integer, can then be modeled by the probability distribution

$$p(k) = \frac{\lambda^k e^{-\lambda}}{k!}.$$

Exercise

- Given a set of observations $\{k_1, \dots, k_N\}$, compute the maximum-likelihood estimate of the parameter λ .
- Use Stirling's approximation and show that we can write the distribution as $p(k) = e^{f(k)}$. What is $f(k)$?

- (c) Interpreting $k \in \mathbb{R}$ as a continuous variable, perform a Taylor expansion of $f(k)$ to second order around its maximum to show that we can approximate $p(k)$ as a normal distribution (don't worry about the normalization prefactor). Report the mean and variance of the normal distribution.
- (d) In which limit does $p(k)$ become (if we allow $k \in \mathbb{R}$ rather than $k \in \mathbb{Z}$) exactly equal to a normal distribution? Explain why in detail.

10 Solution

- (a) The likelihood of the observed data is $L = \prod_{n=1}^N p(k_n)$. Taking the derivative of $\ln L$ with respect to λ gives the maximum-likelihood estimate $\lambda_{\text{MLE}} = \frac{1}{N} \sum_{n=1}^N k_n$.
- (b) Using the Stirling approximation $k! \approx k(\ln k - 1)$ for $k \gg 1$, we get $p(k) \approx e^{f(k)}$, with $f(k) = k \ln \lambda - \lambda - k \ln k + k$.
- (c) Setting $f'(k) = 0$, we find that the maximum occurs at $k = \lambda$. We can then write

$$f(k) = f(\lambda) + \frac{1}{2!} f''(\lambda)(k - \lambda)^2 + O((k - \lambda)^3) = -\frac{1}{2\lambda}(k - \lambda)^2 + O((k - \lambda)^3).$$

From this, we have $p(k) \approx e^{-(k-\lambda)^2/(2\lambda)}$, which is the form of a normal distribution. We can thus read off that the mean and variance are both equal to λ .

- (d) $p(k)$ becomes exactly equal to the normal distribution as $\lambda \rightarrow \infty$. One way to see this is to recall that the calls arriving at our call center are random and uncorrelated, so a random variable $k \sim p(k|\lambda)$ can just as well be written as a sum of random variables $k = \sum_{m=1}^M \tilde{k}_m$, where $\tilde{k}_m \sim p(k|\frac{\lambda}{M})$, with $M \in \mathbb{N}_+$. As $M \rightarrow \infty$ with λ/M remaining finite, the central limit theorem tells us that this sum of random variables will exactly follow a normal distribution. (Answers here may vary, but should invoke the CLT and/or show explicitly that $p(k)$ becomes normal as $\lambda \rightarrow \infty$.)

11 Grading

I propose roughly: 2 points for (a), 2 points for (b), 3 points for (c), 3 points for (d) - but then be sure that the overall grade aligns with 9 or 10 points being an A - “good or exceptional”, 7 or 8 for B “passing but not exceptional”

12 Problem 4 - Mixture of exponential distributions

In the block of code below, we generate data from two exponential distributions: $x_n \sim p(x)$, where

$$p_\alpha(x) = \begin{cases} \lambda_\alpha e^{-\lambda_\alpha x}, & x \geq 0 \\ 0, & \text{else} \end{cases}$$

where $\lambda_\alpha > 0$, and $\alpha \in \{1, 2\}$.

```
[4]: import numpy as np
import matplotlib.pyplot as plt
```

```

# Make some fake data.

n = 10000 # total number of data points

# Parameters for the two distributions
# (we'll use underscores to denote the true parameters, which we'll try to
  ↳infer below):
lambda1_ = 0.3
lambda2_ = 3

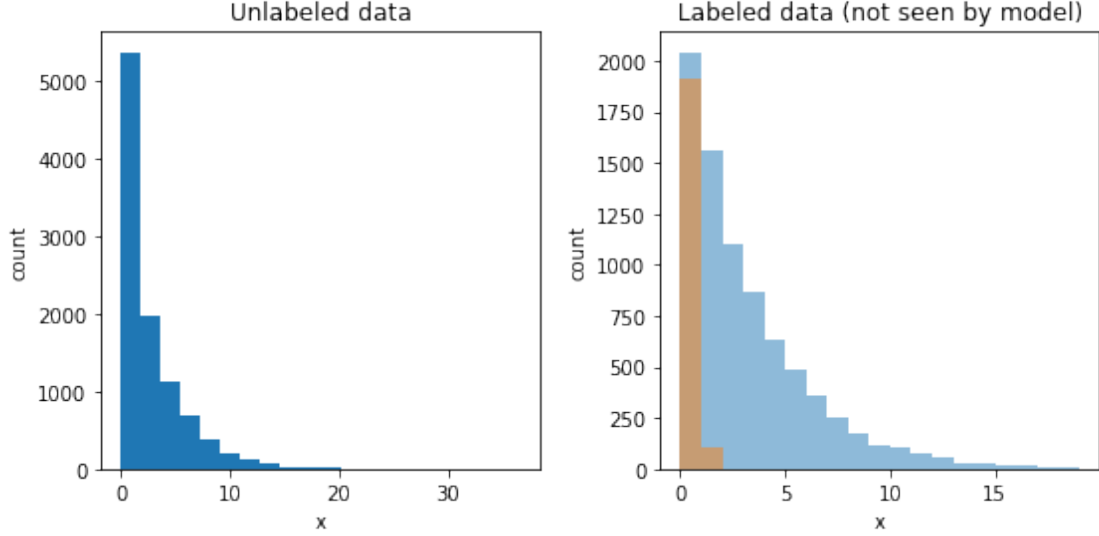
# The mixture probabilities:
pi1_, pi2_ = 0.8, 0.2

# For each data point, choose a cluster that it belongs to and generate its
  ↳location:
x = []
labels = []
for ii in range(n):
    if np.random.rand() < pi1_:
        x.append(np.random.exponential(1/lambda1_))
        labels.append(0)
    else:
        x.append(np.random.exponential(1/lambda2_))
        labels.append(1)

# Plot the data:
plt.figure(figsize=(8,4))
plt.subplot(121)
plt.title('Unlabeled data')
plt.hist(x, bins=20)
plt.xlabel('x')
plt.ylabel('count')

plt.subplot(122)
plt.title('Labeled data (not seen by model)')
plt.hist(np.array(x)[np.array(labels)==0], alpha=0.5, bins=np.arange(20))
plt.hist(np.array(x)[np.array(labels)==1], alpha=0.5, bins=np.arange(20))
plt.xlabel('x')
plt.ylabel('count')
plt.tight_layout()

```

In this exercise, given the unlabeled data, we will use the Expectation Maximization algorithm to try to recover the statistical model that was used to generate this data, inferring the parameters $\Theta = \{\lambda_1, \lambda_2, \pi_1, \pi_2\}$.

Recall that the EM algorithm can be summarized as follows:

1. Choose initial guess for parameters Θ^{old} .
2. Iterate:
 - 2a. E step: Evaluate $p(\mathbf{Z}|\mathbf{X}, \Theta^{\text{old}})$. Concretely, this consists of using Θ^{old} to calculate the *responsibilities* $\gamma(z_{nk})$, where $z_{nk} \in \{0, 1\}$, n is the data point, and k is the latent-variable index.
 - 2b. M step: Using the responsibilities calculated in the E step, let $\Theta^{\text{new}} = \text{argmax}_{\Theta} Q(\Theta, \Theta^{\text{old}})$, where $Q(\Theta, \Theta^{\text{old}}) = \sum_{\mathbf{Z}} p(\mathbf{Z}|\mathbf{X}, \Theta^{\text{old}}) \ln p(\mathbf{X}, \mathbf{Z}|\Theta)$.
 - 2c. Check for convergence of the log-likelihood and/or Θ . If not converged, update $\Theta^{\text{old}} \leftarrow \Theta^{\text{new}}$.

Exercise

- (a) Give the mathematical expressions for the responsibilities $\gamma(z_{nk})$ and the update equations for λ_{α} in terms of $\pi_{1,2}$, $\lambda_{1,2}$, and the data $\{x_n\}_{n=0}^N$.
- (b) Implement the EM algorithm to find the maximum likelihood solution for a mixture of exponentials model with $K = 2$ applied to the data above. Report the inferred values of the parameters Θ . Plot the value of the log likelihood at each step in the iteration and show that it converges. The algorithm should be implemented from scratch, with only basic Numpy functions being used.

13 Solution

(part a): The expressions are

$$\gamma(z_{nk}) = \frac{\pi_k \lambda_k e^{-\lambda_k x_n}}{\sum_{k'=1}^2 \pi_{k'} \lambda_{k'} e^{-\lambda_{k'} x_n}}$$

and

$$\lambda_k = \frac{\sum_{n=1}^2 \gamma(z_{nk})}{\sum_{n=1}^2 \gamma(z_{nk}) x_n}$$

```
[218]: ## Solution (part b) ##

# Initial guesses for parameters:
lambda1, lambda2 = 2., 3.
pi1, pi2 = 0.5, 0.5

n_iter = 50 # number of iterations for the EM algorithm
gamma = np.zeros((n, 2)) # initialize the responsibilities
ll_list = [] # track the log likelihood in each iteration
for ii in range(n_iter):
    # E step:
    for jj in range(n):
        denominator = pi1 * np.exp(-lambda1 * x[jj]) * lambda1 \
            + pi2 * np.exp(-lambda2 * x[jj]) * lambda2
        gamma[jj,0] = pi1 * np.exp(-lambda1 * x[jj]) * lambda1 / denominator
        gamma[jj,1] = pi2 * np.exp(-lambda2 * x[jj]) * lambda2 / denominator

    # M step:
    N1, N2 = np.sum(gamma[:,0]), np.sum(gamma[:,1])
    lambda1 = N1 / (gamma[:,0] @ x)
    lambda2 = N2 / (gamma[:,1] @ x)
    pi1, pi2 = N1/n, N2/n

    # Evaluate LL:
    logp = 0
    for jj in range(n):
        logp += np.log(pi1 * np.exp(-lambda1 * x[jj]) * lambda1
            + pi2 * np.exp(-lambda2 * x[jj]) * lambda2)
    ll_list.append(logp)

# Report the results (note that the 1 and 2 labels may be switched in the
↪generating vs. fitted models):
print('(estimated lambda1, estimated lambda2): ', (lambda1, lambda2))
print('(true lambda1, true lambda2): ', (lambda1_, lambda2_))
print('(estimated pi1, estimated pi2): ', (pi1, pi2))
print('(true pi1, true pi2): ', (pi1_, pi2_))
```

```
# Plot the log likelihood:
plt.figure(figsize=(4,4))
plt.plot(ll_list)
plt.xlabel('Iterations')
plt.ylabel('Log likelihood')
```

```
-----
IndexError                                Traceback (most recent call last)
<ipython-input-218-363a88fb2a6c> in <module>
    11     # E step:
    12     for jj in range(n):
--> 13         denominator = pi1 * np.exp(-lambda1 * x[jj]) * lambda1 \
    14                     + pi2 * np.exp(-lambda2 * x[jj]) * lambda2
    15         gamma[jj,0] = pi1 * np.exp(-lambda1 * x[jj]) * lambda1 / \
↪ denominator

IndexError: index 10 is out of bounds for axis 0 with size 10
```

Note: as you can see on my worksheet I had troubles running this problem's code - also, I had to edit the problem statement's code with different "labeling" code in order to get it to run, I'm not sure why. I hope I did it correctly.

I'd say perhaps 3 marks for (a), 7 for (b), with once again the proviso that a B grade (7 or 8 total) should be the rough "passing threshold" and an A grade (9 or 10 total) should be "excellent work".

14 Problem 5 - Lattice Paths

Let S be the set of lattice paths from $(0,0)$ to $(20, 20)$, taking unit-length steps up and to the right. Consider the probability measure π on S such that the probability of the path P is proportional to $0.9^{A(P)}$, where $A(P)$ is the area between P and the x axis.

Describe a markov chain whose stationary distribution is π . Then use monotone coupling from the past to draw a sample from π ; plot a picture of the sample.

15 Solution

Let $q = 0.99$.

Encode such a set of lattice paths as a sequence of 20 "U" (for "up") and "R" for right. Then, if I've got a path of the form

$p_1 = \dots \text{UR} \dots$

where the UR is in the i th spot, and another identical path which goes like

$p_2 = \dots \text{RU} \dots$

then p_1 encloses one more square than p_2 , so we should have

$$\frac{\pi(p_1)}{\pi(p_2)} = q.$$

The markov chain we want, then, chooses a point i and, if there is an UR or RU there, resamples it randomly with probability $q/(1+q)$ for UR, $1/(1+q)$ for RU. This is the Glauber chain; its stationary distribution is π .

To do one round of monotone coupling from the past:

- pick a starting value of N . - we choose a list of random instructions $(i, \pm 1)$ with the above probabilities.
- 1 means go up in the poset of paths: i.e. make the substring in position i equal to UR if possible.
- -1 means make the substring RU if possible - Apply the sequence of instructions to both the maximal path UUUU...RRRR and the minimal path RRR...UUUU.

Then, if the result of the two sets of instructions is the same, we've sampled a random path.

Otherwise: *prepend* another list of N random instructions to the list, double N , and try again.

The point is to simulate the situation of the chain having been running for a long time, so full marks for this only for this EXACT thing, not-quite-full-marks for small variations of it. Doing basically anything else isn't really doing coupling from the past, but it is running a markov chain - so will likely result in as good a random sample as you'd get from any other MCMC technique.

16 Grading

An A grade (9 or 10) would involve implementing coupling from the past correctly.

A B grade (7 or 8) would involve some error in implementation, which means you're doing some kind of MCMC sampling, yielding plausible results, but not demonstrably bias-free sampling.

```
[134]: q = 0.9

def reset():
    upper_path = "U"*20 + "R"*20
    lower_path = "R"*20 + "U"*20
    return upper_path, lower_path

def new_instruction():
    i = np.random.randint(39)
    if np.random.rand() < 1/(1+q):
        direction = -1
    else:
        direction = 1
    return (i, direction)

def some_instructions(n):
    return [ new_instruction() for trial in range(n) ]

def apply(instruction, state):
```



```
[134]: 'RRUURRRRRRRUURUURUURRUUUURURUUUUURURU'
```

[209]: 340

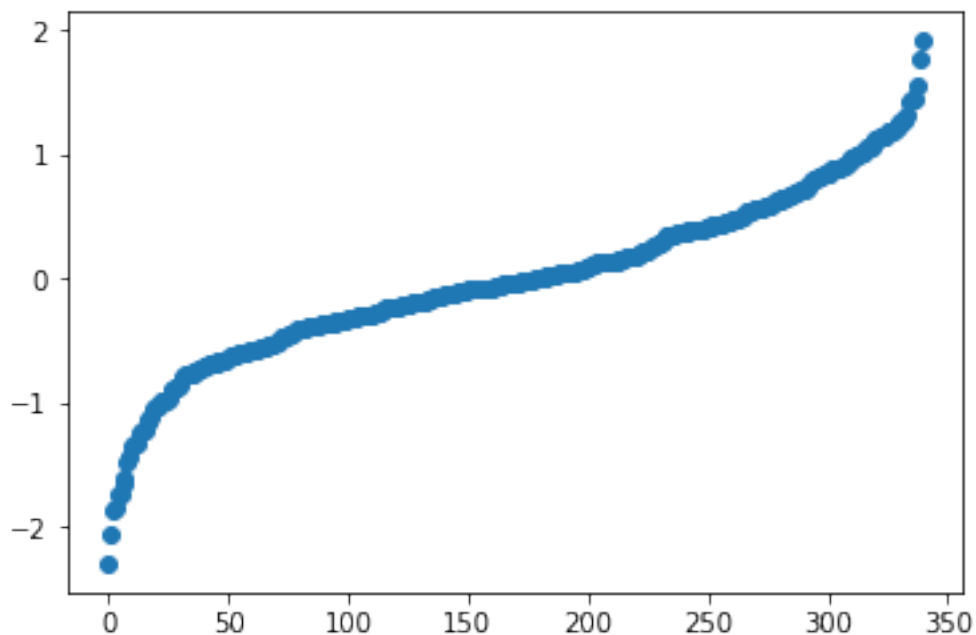
So, that's how many points we're going to sample. Then, we'll do rejection sampling, as per wikipedia. Choose a point x at random, and an "acceptance value" $\tau \sim \text{Uniform}(0, 1)$; accept the point if $\tau < \frac{f(x)}{\lambda} = \frac{100e^{-x^2}}{\lambda}$.

```
[210]: L = []
while(len(L)<n):
    candidate = np.random.uniform(-5,5)
    tau = np.random.uniform(0,1)
    threshold = 100*np.exp(-candidate**2) / llambda
    if tau < threshold:
        L.append(candidate)
```

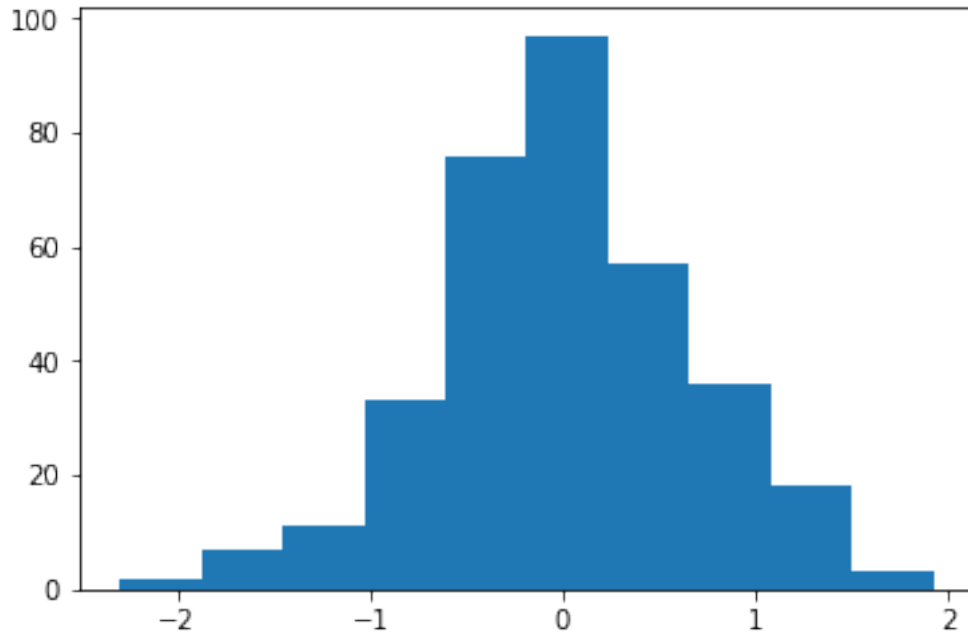
Here's a picture of the points in the sample, sorted. Looks like they're more or less between -2 and 2. And a histogram, which looks like a bell curve, that being the shape of the graph of f .

```
[211]: L.sort()
x = np.arange(n)
sample = np.asarray(L)
plt.scatter(x,sample)
```

[211]: <matplotlib.collections.PathCollection at 0x71cf9bf4eda0>



```
[212]: (counts, x_coords) = plt.hist(sample)[:2]
```



19 Solution - inference

To infer what f is: well, there's a variety of things one could say, but the simplest thing to say is that the graph of f should be basically identical to the above histogram?

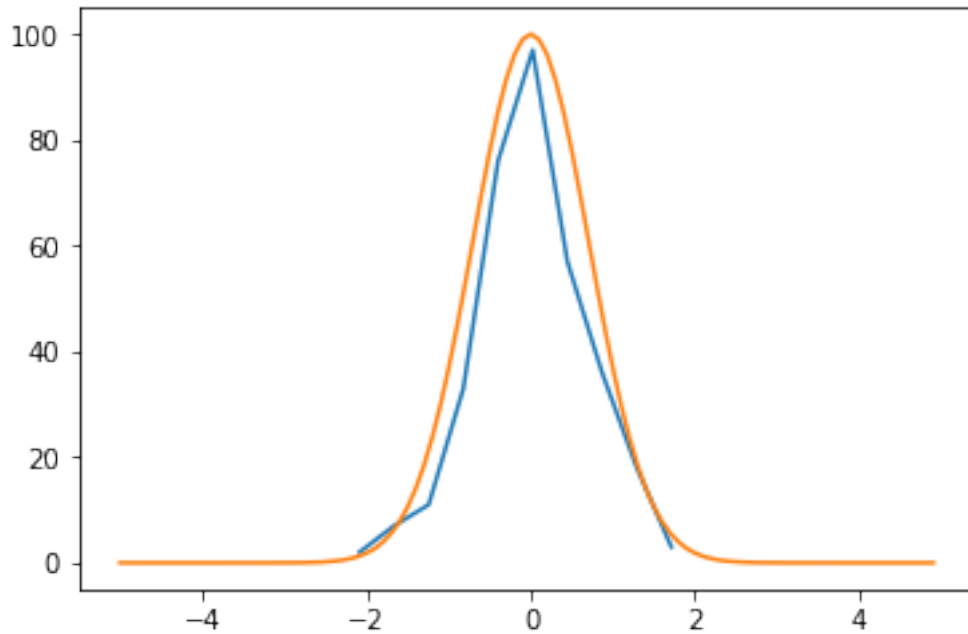
Somehow the hardest part of this problem is to convert the bar graph-style histogram into a curve plot for f . The data from matplotlib's histogram has its x coordinates at the *endpoints* of the bars, irritatingly.

```
[215]: x = np.asarray([(x_coords[i] + x_coords[i+1])/2 for i in
↪range(len(x_coords)-1)])
```

```
[216]: smooth_x = np.arange(-5, 5, 0.1)

plt.plot(x, counts)
plt.plot(smooth_x, 100*np.exp(-smooth_x**2))
```

```
[216]: [<matplotlib.lines.Line2D at 0x71cfa1d81780>]
```

Of course, by doing more runs, we can do better inference, have smaller bins, get a more refined bell curve and so forth.

20 Grading

I'd say about 7 points for the “sampling” part, 3 parts for the “inference” part; a grade of A (9 or 10) involves doing all the above stuff correctly; a grade of B (7 or 8) means that perhaps something is missing - incorrect computation of the number of points, for instance, or no description of how to infer the intensity function from a sample. I'd take basically any description of how to approximate the intensity, based on the sample.